Bring everything up

Internal Developer Platform runbook. From nothing on AWS to a working https://idplatform.tech. About 35 minutes, mostly waiting.

Run every command in **Git Bash**, starting from the project folder. Do the steps in order.

```
cd ~/OneDrive/Desktop/IDP\ Project
```

Before you start

Tools you need: AWS CLI, Terraform, kubectl, Helm, Git Bash. **Check you're on the right AWS account:**

```
aws sts get-caller-identity --query Account --output text
```

> Expected: `516982327726`

These files are not in Git. They exist only on your PC, so back them up. Copy the shape from `k8s-secret.example.yml` if you ever lose one.

File	Holds
`k8s/secret.yml`	Database, Git, Redis and ArgoCD passwords, and the GitHub login keys from the **IDP Prod** OAuth App
`k8s/database/secret.yml`	Postgres name, user and password
`idp-backend/.env`	`GIT_PASSWORD` and the GitHub login keys from the **IDP Local** OAuth App

---

1. Create the infrastructure about 20 minutes

```
cd terraform
helm repo update
terraform init
terraform plan -out=tfplan
terraform apply tfplan
cd ..
```

> Expected at the end: `Apply complete! Resources: 66 added, 0 changed, 0 destroyed.`

Keep the PC awake and online. If it stops with a network error such as `no such host`, nothing is broken: run `terraform plan -out=tfplan` and `terraform apply tfplan` again, and it continues from where it stopped.

---

2. Connect kubectl to the new cluster

Every new cluster gets a new address, so this is needed after every apply.

```
aws eks update-kubeconfig --name idp-cluster --region eu-north-1
kubectl get nodes
```

> Expected: 3 nodes, all `Ready`.

---

3. Give ArgoCD access to the Git repository

```
GP=$(tr -d '\r' < idp-backend/.env | sed -n 's/^GIT_PASSWORD=//p')
kubectl create secret generic repo-idp -n argocd \
  --from-literal=type=git \
  --from-literal=url=https://github.com/AminNeji/Internal-Developer-Platform.git \
  --from-literal=username=AminNeji \
  --from-literal=password="$GP"
kubectl label secret repo-idp -n argocd \
  argocd.argoproj.io/secret-type=repository
```

---

4. Add the secrets and start the platform

```
kubectl apply -f k8s/namespace.yml
kubectl apply -f k8s/secret.yml
kubectl apply -f k8s/database/secret.yml
kubectl apply -f argocd/platfrom-app.yml
```

ArgoCD now deploys everything in `k8s/` from Git. The backend and frontend show `ImagePullBackOff` until step 6. That's expected.

---

5. Create the ArgoCD token for the backend

The token never expires, but it's tied to this cluster, so a new cluster needs a new one. This block creates it, saves it in `k8s/secret.yml` and applies it. Paste the whole block at once.

```
kubectl port-forward svc/argocd-server -n argocd 8089:80 > /dev/null &
PF=$!; sleep 5
PW=$(kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d)
ADMIN=$(curl -s -X POST localhost:8089/api/v1/session \
  -H 'Content-Type: application/json' \
  -d "{\"username\":\"admin\",\"password\":\"$PW\"}" \
  | sed -n 's/.*"token":"\([^"]*\)".*/\1/p')
TOKEN=$(curl -s -X POST localhost:8089/api/v1/account/idp-backend/token \
  -H "Authorization: Bearer $ADMIN" \
  -H 'Content-Type: application/json' -d '{}' \
  | sed -n 's/.*"token":"\([^"]*\)".*/\1/p')
kill $PF
echo "token length: ${#TOKEN}"
[ -n "$TOKEN" ] && sed -i "s|^\(  ARGOCD_TOKEN:\).*|\1 \"$TOKEN\"|" k8s/secret.yml
kubectl apply -f k8s/secret.yml
```

> Expected: a token length around 250 and `secret/app-secret configured`. If the length is 0, wait a minute for ArgoCD to finish starting and paste the block again.

6. Rebuild the images about 8 minutes

A destroy empties the image registry, so the images have to be built again.

1. Go to github.com/AminNeji/Internal-Developer-Platform/actions
2. Open the most recent run on `dev`
3. Click **Re-run all jobs** and wait for it to turn green

GitHub only allows re-runs for 30 days. If it refuses, edit any file outside `k8s/` and `environments/` (a line in the README is enough), then commit and push to `dev`.

7. Check that it works about 5 minutes after CI finishes

```
kubectl get pods -n idp-project
kubectl get ingress -n idp-project
aws route53 list-resource-record-sets --output text \
  --hosted-zone-id Z043797928Y1HP8AB44R8 \
  --query "ResourceRecordSets[?Type=='A'].[Name,AliasTarget.DNSName]"
curl -s -o /dev/null -w '%{http_code}\n' https://idplatform.tech/api/me
```

> Expected: 4 pods `Running`, an A record for `idplatform.tech` (ExternalDNS creates it within a minute or two, nothing to do by hand), and `401` from the last line, which means login is protecting the API.

The backend takes about a minute to start, so a brief `502` or `503` is normal.

Open **https://idplatform.tech**, sign in with GitHub and create a test environment with the image `nginx:alpine`, port `80`. It should turn Healthy in about 2 minutes, with a link on its own subdomain.

If something goes wrong

What you see	What to do
`no such host` from kubectl	Step 2 was skipped. kubectl is pointing at the deleted cluster.
`no such host` or a timeout from Terraform	A network blip. Run plan and apply again.
`Error acquiring the state lock`	An earlier run was interrupted. Run `terraform force-unlock -force <ID>` with the ID it prints, then try again.
Helm errors about a chart or repository	Run `helm repo update`, then plan and apply again.
Pods stuck in `ImagePullBackOff`	Step 6 hasn't finished. If CI is green, wait 3 minutes for ArgoCD to notice.
Backend in `CrashLoopBackOff`	Run `kubectl logs -n idp-project deploy/idp-backend-deployment --tail=30`. `Could not resolve placeholder` means a key is missing from `k8s/secret.yml`. Add it, apply the file and run the restart command below.
Environments show `Unknown`, or creating one fails	The ArgoCD token is wrong. Repeat step 5, then restart: `kubectl rollout restart deploy/idp-backend-deployment -n idp-project`

GitHub says the redirect URI is invalid	The **IDP Prod** OAuth App's callback must be exactly `https://idplatform.tech/api/login/oauth2/code/github`
"That GitHub account isn't allowed"	Add the username to `IDP_ALLOWED_USERS` in `k8s/configmap.yml`, comma-separated. Push it, then restart the backend.
The site name doesn't resolve	Run `kubectl logs -n kube-system deploy/external-dns --tail=20`. It only manages records it created, so delete any A record for that name made by hand and it will create its own.
Terraform says `VcpuLimitExceeded`	This account allows 8 vCPUs, so a node group can't be replaced while the old one still runs. Scale down or delete the old node group first.
`release version 21 not supported` when building the backend locally	`JAVA_HOME` points to JDK 17. Point it to JDK 21.

What it costs while running

About **$0.30 an hour, or $7 a day** (rough figures, check your AWS bill): EKS, 3 small nodes, the NAT gateway, the load balancer and Redis. That's why the platform gets destroyed when it isn't in use. See the destroy runbook.